# Contenido

Práctica 6. Despliegue de una aplicación Python con Flask ..... 2
Objetivos ..... 2
Arquitectura inicial ..... 2
¿Qué es Flask? ..... 2
Actividad 1. Comprobar Python ..... 3
Actividad 2. Crear entorno de trabajo ..... 3
Actividad 3. Instalar Flask ..... 3
Actividad 4. Crear aplicación Python ..... 5
Actividad 5. Ejecutar la aplicación ..... 6
Actividad 6. Acceder desde Windows ..... 7
Comparación PHP vs Flask ..... 7
Actividad 7. Aplicación dinámica ..... 7
Actividad 8. Comprobar puertos ..... 8
Conclusiones ..... 9

# Práctica 6. Despliegue de una aplicación Python con Flask

## Objetivos

Al finalizar la práctica el seremos capaces de:

- Instalar Python y Flask.
- Crear una aplicación web básica.
- Ejecutar aplicaciones Flask.
- Acceder desde el navegador.
- Comprender las diferencias entre PHP y Python.
- Desplegar aplicaciones Python en un servidor Linux.

Los RAs cubiertos son principalmente el **RA3**, aunque también empieza a preparar contenidos posteriores.

- **RA3.c** Se han desplegado aplicaciones PHP, Python y Java (Tomcat). Completo para Python.
- **RA3.a** (parcialmente) Se han instalado y configurado servidores Apache y Nginx. La siguiente práctica sí lo cubrirá completamente.
- **RA3.e** (introducción). Se han optimizado configuraciones para diferentes cargas de trabajo.
- **RA6.e** Se han creado manuales de instalación y configuración. Documentando la instalación de Flask, creación de app.py y pruebas realizadas.

## Arquitectura inicial

Partimos de **Windows → Apache → PHP**

Y añadiremos **Windows → Flask → Python**

## ¿Qué es Flask?

Flask es un microframework de desarrollo web para Python que permite crear aplicaciones web y servicios HTTP de forma sencilla. Gestiona peticiones de los clientes, ejecuta código Python en el servidor y genera respuestas que son enviadas al navegador. Es uno de los frameworks más utilizados para iniciarse en el desarrollo web con Python.

¿Por qué se llama microframework? Porque incluye solamente lo esencial. No trae por defecto:

- Sistema de autenticación.
- Panel de administración.
- ORM completo.
- Gestión avanzada de usuarios.

Eso permite:

- Aprender más fácilmente.
- Entender mejor el funcionamiento interno.

- Añadir únicamente los componentes necesarios.

Flask se utiliza para desarrollar:

- Aplicaciones web.
- APIs REST.
- Paneles de administración.
- Herramientas internas de empresa.
- Servicios web.

Muchas organizaciones utilizan Python y Flask para crear sus aplicaciones backend.

Ventajas de Flask:

- Fácil de aprender.
- Muy poco código.
- Ideal para comprender la arquitectura cliente-servidor.
- Permite conectar con MariaDB.
- Se integra perfectamente con Apache y Nginx.
- Introduce al desarrollo web con Python.

## Actividad 1. Comprobar Python

Verificar instalación: **python3 --version**

Resultado esperado: **Python 3.x**

Comprobar pip: **pip3 –version**

**pip3** es el gestor de paquetes de Python 3. Permite instalar, actualizar y administrar librerías y módulos externos necesarios para desarrollar aplicaciones Python, como **Flask**, Django o MySQL Connector.

## Actividad 2. Crear entorno de trabajo

Crear directorio: **mkdir ~/flask**

Entrar: **cd ~/flask**

```
profesor@srv-daw:~$ mkdir ~/flask
profesor@srv-daw:~$ cd ~/flask
```

## Actividad 3. Instalar Flask

Instalamos: **pip3 install flask**

```
profesor@srv-daw:~/flask$ pip3 install flask
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have python3-full installed.

    If you wish to install a non-Debian packaged Python application,
    it may be easiest to use pipx install xyz, which will manage a
    virtual environment for you. Make sure you have pipx installed.

    See /usr/share/doc/python3.14/README.venv for more information.

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can ove
rride this, at the risk of breaking your Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
profesor@srv-daw:~/flask$
```

Este error se debe a que Ubuntu protege la instalación de Python del sistema para evitar que **pip3** rompa paquetes instalados mediante apt. En entornos profesionales no se instala Flask directamente en el sistema, sino dentro de un **entorno virtual (venv)**.

Además tenemos instalado Python 3.14, pero **no está instalado el paquete específico python3.14-venv**, que es el que permite crear entornos virtuales.

Ejecutamos: **sudo apt update**

Después: **sudo apt install python3.14-venv -y**

Cuando termine:

**cd ~/flask**

**python3 -m venv venv**

Debería crear correctamente la carpeta:

```
~/flask
│
├── app.py
└── venv
```

```
profesor@srv-daw:~/flask$ cd ~/flask
profesor@srv-daw:~/flask$ python3 -m venv venv
profesor@srv-daw:~/flask$
```

Activamos el entorno virtual: **source venv/bin/actívate**

Veremos algo parecido a:

**(venv) profesor@srv-daw:~/flask$**

```
profesor@srv-daw:~/flask$ source venv/bin/activate
(venv) profesor@srv-daw:~/flask$
```

Instalamos Flask ya dentro del entorno virtual: **pip install flask**

Verificamos: **pip3 show flask**

Debería aparecer: **Flask**

```
(venv) profesor@srv-daw:~/flask$ pip3 show flask
Name: Flask
Version: 3.1.3
Summary: A simple framework for building complex web applications.
Home-page:
Author:
Author-email:
License-Expression: BSD-3-Clause
Location: /home/profesor/flask/venv/lib/python3.14/site-packages
Requires: blinker, click, itsdangerous, jinja2, markupsafe, werkzeug
Required-by:
(venv) profesor@srv-daw:~/flask$
```

Verificamos la versión de Python: **python3 --version**

```
(venv) profesor@srv-daw:~/flask$ python3 --version
Python 3.14.4
(venv) profesor@srv-daw:~/flask$
```

Eso significa que:

- El entorno virtual se ha creado correctamente.
- El entorno virtual está activado.
- Ya no estamos utilizando el Python del sistema.
- Ya puedes instalar Flask correctamente.

Ahora tenemos un entorno virtual

**Sistema Operativo Ubuntu → Python 3.14.4 → Entorno Virtual (venv) → Flask**

Ventajas:

- Cada aplicación tiene sus propias librerías.
- Evitamos conflictos entre proyectos.
- No modificamos Python del sistema.
- Es la forma profesional de trabajar con Python.

**¿Por qué usamos un entorno virtual en lugar de instalar Flask directamente en Ubuntu?**

Porque permite aislar las dependencias de cada proyecto, evitar conflictos entre versiones y proteger la instalación principal de Python del sistema operativo.

## Actividad 4. Crear aplicación Python

En el entorno virtual accede a la carpeta: **cd ~/flask**

Y crea: **nano app.py**

```
drwxrwxr-x 5 profesor profesor 4096 Oct 20 11:10 venv
(venv) profesor@srv-daw:~/flask$ cd ~/flask
(venv) profesor@srv-daw:~/flask$ nano app.py
```

Introduce el contenido:

```
from flask import Flask


app = Flask(__name__)

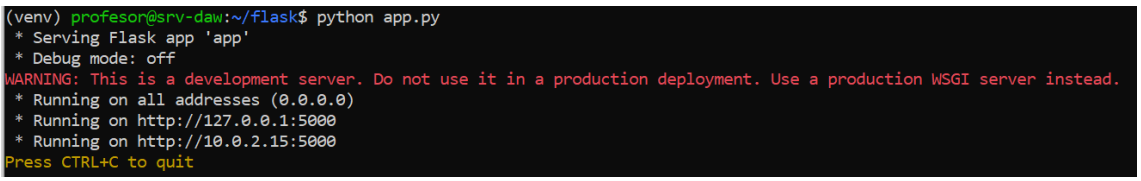

@app.route("/")
def inicio():
return "<h1>Hola desde Flask</h1>"


if __name__ == "__main__":
app.run(
host="0.0.0.0",
port=5000
)
```

Guardar.

## Actividad 5. Ejecutar la aplicación

Con el entorno virtual aún activo ejecutamos: **python app.py**




La captura demuestra que **la aplicación Flask ya está funcionando correctamente**.

**Running on all addresses (0.0.0.0)**

**Flask está ejecutándose**. El framework Flask ha cargado correctamente la aplicación: app = Flask(__name__) y ha arrancado el servidor web interno.

**Python está escuchando peticiones HTTP.** La aplicación está funcionando como servidor web. Actualmente escucha en Puerto 5000/TCP

Por eso podremos acceder mediante: **http://192.168.56.111:5000**

Flask está escuchando en todas las interfaces. Esta línea 0.0.0.0 significa aceptar conexiones desde cualquier interfaz de red. Si hubiésemos usado host="127.0.0.1" solamente funcionaría desde el propio Ubuntu. Gracias a host="0.0.0.0" podremos acceder desde Windows.

Flask también muestra http://10.0.2.15:5000 que corresponde al adaptador NAT. Nuestra IP Host-Only es 192.168.56.111

## Actividad 6. Acceder desde Windows

Abrir: **http://192.168.56.111:5000**

Resultado esperado:

**Hola desde Flask**




## Comparación PHP vs Flask

**PHP** Cliente → Apache → PHP

**Flask** Cliente → Flask → Python

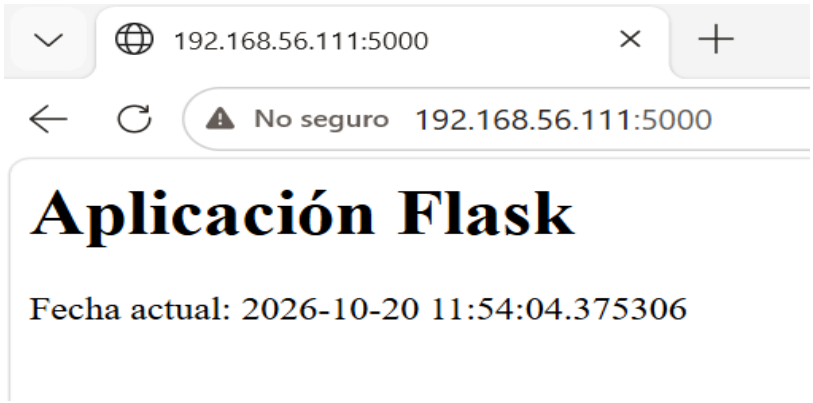
## Actividad 7. Aplicación dinámica

Modificamos **app.py** con el siguiente Código. Ojo que la función **inicio**() esta **indentada (tabulada)**:

```
from flask import Flask
from datetime import datetime
app = Flask(__name__)
@app.route("/")
def inicio():
  return f"""
  <h1>Aplicación Flask</h1>
  <p>Fecha actual: {datetime.now()}</p>
  """
if __name__ == "__main__":
  app.run(
    host="0.0.0.0",
    port=5000
  )
```

Actualizar navegador: **F5**




**¿Por qué cambia la fecha en cada actualización?**

Porque Python vuelve a ejecutar el código en cada petición HTTP.

## Actividad 8. Comprobar puertos

Mientras Flask está ejecutándose en el servidor ejecutamos: **ss -tulpn**

Debe aparecer: **5000**

```
profesor@srv-daw:~$ ss -tulpn
Netid       State       Recv-Q      Send-Q                Local Address:Port          Peer Address:Port      Process
udp         UNCONN      0           0                        127.0.0.1:323              0.0.0.0:*
udp         UNCONN      0           0                       127.0.0.54:53               0.0.0.0:*
udp         UNCONN      0           0                    127.0.0.53%lo:53               0.0.0.0:*
udp         UNCONN      0           0                192.168.56.111%enp0s8:68           0.0.0.0:*
udp         UNCONN      0           0                    10.0.2.15%enp0s3:68            0.0.0.0:*
udp         UNCONN      0           0                            [::1]:323                 [::]:*
tcp         LISTEN      0           4096                 127.0.0.53%lo:53               0.0.0.0:*
tcp         LISTEN      0           4096                    127.0.0.54:53               0.0.0.0:*
tcp         LISTEN      0           80                      127.0.0.1:3306              0.0.0.0:*
tcp         LISTEN      0           128                       0.0.0.0:5000              0.0.0.0:*          users:(("python",pid=3503,fd=3))
tcp         LISTEN      0           4096                      0.0.0.0:22                0.0.0.0:*
tcp         LISTEN      0           511                             *:80                      *:*
tcp         LISTEN      0           4096                         [::]:22                   [::]:*
tcp         LISTEN      0           511                             *:443                     *:*
profesor@srv-daw:~$
```

Comprobar: **nmap localhost**

```
profesor@srv-daw:~$ nmap localhost
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-20 11:57 +0000
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00033s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 995 closed tcp ports (conn-refused)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
443/tcp  open  https
3306/tcp open  mysql
5000/tcp open  upnp

Nmap done: 1 IP address (1 host up) scanned in 0.20 seconds
profesor@srv-daw:~$
```

Resultado esperado:

**22/tcp open ssh**

**80/tcp open http**

**443/tcp open https**

**5000/tcp open upnp**

**(o identificado como servicio desconocido según la versión de Nmap).**

**Analizamos el flujo**

Cuando visitamos: **http://192.168.56.111:5000**

Ocurre **Navegador → HTTP → Flask → Python → Genera HTML → Navegador**

## Conclusiones

Al finalizar la práctica se ha comprobado que:

- Python puede utilizarse para desarrollar aplicaciones web.
- Flask permite crear aplicaciones web ligeras y rápidas.
- Las aplicaciones Flask escuchan por defecto en el puerto 5000.
- Python genera contenido dinámico del mismo modo que PHP.
- Un servidor Linux puede ejecutar simultáneamente Apache, PHP y Flask.

La siguiente práctica será: **Práctica 7. Publicar Flask mediante Apache o Nginx utilizando un Reverse Proxy.**

Ahí cubriremos completamente:

- RA3.a Configuración de Apache y Nginx.
- RA3.c Despliegue de aplicaciones Python.
- RA3.e Optimización y publicación en entornos de producción.

Y veremos una arquitectura profesional real:

**Cliente → HTTPS → Apache o Nginx → Reverse Proxy → Flask → Python**